Session Management

HTTP is a **stateless protocol**.
That means:

- Browser sends request → server gives response
- Server **does not remember** previous request

Example:

You login on a website.
When you open next page, server should know:

"This user already logged in"

To remember the user, we use **Session Management**.

Types of Session Management

1. Client Side

Data stored in browser.

Methods:

- Cookies
- URL Rewriting
- Hidden Form Field

---

2. Server Side

Data stored on server.

Method:

- HttpSession

Real Flow of Session

User Login
↓
Server creates session
↓
Browser gets JSESSIONID cookie
↓
User sends next request
↓
Browser automatically sends cookie
↓
Server identifies same user session

---

1. HttpSession (Server Side)

Server stores data for a user.

Think like:

Server gives every user a unique locker number.

Create Session

HttpSession session = request.getSession();

- Creates new session if not available
- Returns existing session if already present

Store Data in Session

session.setAttribute("user", "admin");

Here:

- "user" → key
- "admin" → value

Stored on server.

Get Data from Session

String user = (String) session.getAttribute("user");

Retrieves stored value.

Destroy Session

Correct method:

session.invalidate();

Session Timeout

Session automatically expires after some time.

Using Java Code

session.setMaxInactiveInterval(300);

300 seconds = 5 minutes

If user inactive for 5 min → session destroyed.

Using web.xml

```
<session-config>
   <session-timeout>10</session-timeout>
</session-config>
```

10 minutes timeout.

JSESSIONID Cookie

When session is created:

Server automatically creates a cookie:

JSESSIONID = A1B2C3

Browser stores it.

Next request:

Browser sends this cookie back.

Server uses it to identify session.

---

2. Cookies (Client Side)

Cookies are small data stored in browser.

Create Cookie

```
Cookie cookie = new Cookie("user", "admin");
response.addCookie(cookie);
```

Stores cookie in browser.

Read Cookies

```
Cookie[] cookies = request.getCookies();
```

Gets all cookies from browser.

Difference Between Session and Cookie

Session	Cookie
Stored on server	Stored in browser
More secure	Less secure
Uses JSESSIONID	Directly stores data
Bigger data possible	Small data only

---

3. URL Rewriting

Data added in URL.

Example:

/dashboard?JSESSIONID=343434

Used when cookies are disabled.

Problem:

- Visible in URL
- Less secure

---

4. Hidden Form Field

Hidden input stores data inside form.

<input type="hidden" name="user" value="admin">

User cannot see on page directly.

Used to send data page to page.

Simple Real-Life Example

Without Session

Login page → dashboard

Server forgets user.

User must login again and again.

With Session

Login → session created

Now server remembers user until logout or timeout.

Small Login Example

Login Servlet

```
HttpSession session = request.getSession();

session.setAttribute("user", "Rishabh");
```

Dashboard Servlet

```
HttpSession session = request.getSession();

String user = (String) session.getAttribute("user");

out.println("Welcome " + user);
```

Logout Servlet

```
HttpSession session = request.getSession();

session.invalidate();
```

Easy One-Line Definitions

Session

Server-side memory for a user.

Cookie

Small browser storage.

JSESSIONID

Unique session ID stored in cookie.

URL Rewriting

Passing session ID in URL.

Hidden Field

Hidden form data transfer.

JSP

JSP (JavaServer Pages)

JSP is a **server-side technology** used to create dynamic web pages using Java.

Simple Meaning

HTML alone gives static pages.

JSP allows:

HTML + Java

together.

So page can become dynamic.

Example

Without JSP:

<h1>Welcome User</h1>

Same output for everyone.

With JSP:

<h1>Welcome <%= username %></h1>

Different user sees different name.

Client Side vs Server Side

HTML	JSP
Client Side	Server Side
Runs in browser	Runs on server
Static	Dynamic
No Java code	Java supported

Why JSP Preferred?

Because JSP combines:

HTML (UI)
+
Java (Logic)

in one file.

Internal Working of JSP

Step 1

Browser requests:

login.jsp

Step 2

Server converts JSP into Servlet internally.

Step 3

Java code executes.

Step 4

HTML + Java output combined.

Step 5

Final HTML sent to browser.

JSP Scripting Elements

Used to write Java inside JSP.

There are 3 types:

Type	Purpose
Scriptlet Tag	Write Java code
Expression Tag	Print output
Declaration Tag	Declare variables/methods

---

1. Scriptlet Tag

Syntax:

<% %>

Used for Java code.

Example:

```
<%
int a = 10;
int b = 20;
%>
```

Internally

This code goes inside:

_jspService()

method of generated servlet.

Easy Meaning

Whenever request comes:

- _jspService() runs
- scriptlet code executes

Multiple Scriptlets

All scriptlets combine inside same _jspService().

Example:

```
<% int a = 10; %>

<% int b = 20; %>
```

Internally:

```
public void _jspService(){

  int a = 10;

  int b = 20;
}
```

---

2. Expression Tag

Syntax:

```
<%=  %>
```

Used to print output.

Example:

```
<%= "Hello" %>
```

Output:

Hello

Internally

Converted into:

out.print("Hello");

Example

<%= 10 + 20 %>

Output:

30

---

3. Declaration Tag

Syntax:

<%! %>

Used to declare:

- variables
- methods
- classes

Example

```
<%!
int a = 10;

public int square(int n){
  return n*n;
}
%>
```

Difference Between Scriptlet and Declaration

Scriptlet

Goes inside:

_jspService()

Runs for every request.

Declaration

Goes outside _jspService().

Becomes class-level member.

JSP Directives

Directives give instructions to JSP container.

Syntax:

<%@ %>

Types of Directives

Directive	Purpose
Page Directive	Page settings
Include Directive	Include file
Taglib Directive	JSTL/custom tags

---

1. Page Directive

Example:

<%@ page import="java.util.Date" %>

Used for:

- import
- session
- language
- error page etc.

Common Attributes

Import

<%@ page import="java.util.Date" %>

Imports Java class.

Session

<%@ page session="true" %>

Enables session.

---

isErrorPage

<%@ page isErrorPage="true" %>

Makes page error page.

---

2. Include Directive

Syntax:

<%@ include file="header.jsp" %>

Includes file during translation time.

Used for:

- navbar
- header
- footer

---

3. Taglib Directive

Used for JSTL/custom tags.

Example:

<%@ taglib uri="" prefix="c" %>

JSP Action Tags

Special JSP tags for actions.

Syntax:

<jsp:tagname>

Include Action Tag

<jsp:include page="header.jsp"/>

Includes file at request time.

Forward Action Tag

<jsp:forward page="home.jsp"/>

Forwards request to another page.

---

useBean

Creates JavaBean object.

<jsp:useBean id="obj" class="com.demo.User"/>

---

setProperty

Sets bean values.

<jsp:setProperty name="obj" property="name"/>

---

getProperty

Gets bean value.

<jsp:getProperty name="obj" property="name"/>

JSP Custom Tags

User-defined tags.

Used to avoid Java code in JSP.

MVC Architecture

MVC means:

Part	Meaning
Model	Data/business logic
View	JSP page
Controller	Servlet

Flow

Browser
↓
Servlet (Controller)
↓
Business Logic
↓
JSP (View)

Expression Language (EL)

Simpler way to access data.

Instead of:

<%= session.getAttribute("user") %>

Use:

${sessionScope.user}

Cleaner and easier.

JSTL

JSP Standard Tag Library.

Provides ready-made tags.

Used for:

- loops

- conditions
- formatting

Example

<c:if test="">

Implicit Objects in JSP

Automatically available objects.

No need to create manually.

Important Implicit Objects

Object	Purpose
request	Client request data
response	Send response
session	Session handling
application	Whole application data
out	Print output
config	Servlet config
pageContext	JSP page info
exception	Error handling

---

request

Gets form data.

request.getParameter("email");

---

response

Send redirect.

response.sendRedirect("home.jsp");

---

session

Store user data.

session.setAttribute("user","admin");

---

out

Print output.

out.print("Hello");

---

pageContext

Access JSP-related objects.

---

exception

Used in error pages.

Session Validation

Checks whether user logged in.

Example:

```
<%
if(session.getAttribute("user") == null){
   response.sendRedirect("login.jsp");
}
%>
```

Final Simple Summary

JSP

HTML + Java

used to create dynamic web pages.

Scriptlet

Write Java code

Expression

Print output

Declaration

Declare variables/methods

Directives

Instructions to JSP container

Action Tags

Perform actions like include/forward

EL

Easy way to access data

JSTL

Ready-made JSP tags

MVC

Servlet controls
JSP displays